

1. WAP using dynamic array to perform the following:

- Find max and min in the array
- Find the position of a key in the array
- Find the even and odd elements in the array

Source Code:

```
#include <stdio.h>
#include <stdlib.h>

void maxmin(int *ptr, int len);
void pos(int *ptr, int len);
void evenodd(int *ptr, int len);

int main(){
    int len;
    printf("Enter array size : ");
    scanf("%d", &len);

    int *ptr = (int*) malloc(len * sizeof(int));

    printf("Enter elements of array : ");
    for (int i=0; i<len; i++){
        scanf("%d", ptr+i);
    }

    while(1){
        int op;
        printf("\nOperation Menu: \n");
        printf("1. Find Min & Max\n");
        printf("2. Position of key\n");
        printf("3. Even & odd Elements\n");
        printf("Enter your choice : ");
        scanf("%d", &op);

        switch(op) {
            case 1: maxmin(ptr, len); break;
            case 2: pos(ptr, len); break;
            case 3: evenodd(ptr, len); break;
            default: printf("Invalid Operation\n"); break;
        }
    }

    free(ptr);
}
```

```

    return 0;
}

void maxmin(int* ptr, int len){
    int max = *ptr;
    for (int i=0; i<len; i++){
        if (max < *(ptr+i)){
            max = *(ptr+i);
        }
    }

    int min = *ptr;
    for (int i=0; i<len; i++){
        if (min > *(ptr+i)){
            min = *(ptr+i);
        }
    }

    printf("Max Element: %d\n", max);
    printf("Min Element: %d\n", min);
}

void pos(int* ptr, int len){
    int key;
    printf("Enter the key : ");
    scanf("%d", &key);
    for (int i=0; i<len; i++){
        if (key == *(ptr+i)){
            printf("Position : %d\n", i);
        }
    }
}

void evenodd(int* ptr, int len){
    printf("Even Numbers: ");
    for (int i=0; i<len; i++){
        if (*(ptr+i)%2 == 0){
            printf("%d ", *(ptr+i));
        }
    }
    printf("\n");
    printf("Odd Numbers: ");
    for (int i=0; i<len; i++){

```

```
        if (*(ptr+i)%2 != 0){
            printf("%d ", *(ptr+i));
        }
    }
    printf("\n");
}
```

Output:

```
Enter array size : 5
Enter elements of array : 2 5 10 5 10
```

```
Operation Menu:
1. Find Min & Max
2. Position of key
3. Even & odd Elements
Enter your choice : 1
Max Element: 10
Min Element: 2
```

```
Operation Menu:
1. Find Min & Max
2. Position of key
3. Even & odd Elements
Enter your choice : 2
Enter the key:5
Position : 1
Position : 3
```

```
Operation Menu:
1. Find Min & Max
2. Position of key
3. Even & odd Elements
Enter your choice : 3
Even Numbers: 2 10 10
Odd Numbers: 5 5
```

2. WAP to find the largest number and counts the occurrence of the largest number in a dynamic array of n integers using a single loop.

Source Code:

```
#include <stdio.h>
#include <stdlib.h>

void count_find_max(int* ptr, int len);

int main(){
    int len;
    printf("Enter array size : ");
    scanf("%d", &len);

    int *ptr = (int*) malloc(len * sizeof(int));

    printf("Enter elements of array : ");
    for (int i=0; i<len; i++){
        scanf("%d", ptr+i);
    }

    count_find_max(ptr, len);
    free(ptr);
    return 0;
}

void count_find_max(int* ptr, int len){
    int max = *ptr;
    for (int i=0; i<len; i++){
        if (max < *(ptr+i)){
            max = *(ptr+i);
        }
    }

    int count = 0;
    for (int i=0; i<len; i++){
        if (max == *(ptr+i)){
            count++;
        }
    }

    printf("Largest Element: %d\n", max);
}
```

```
printf("No. of Occurances: %d\n", count);  
}
```

Output:

```
Enter array size : 5  
Enter elements of array : 1 3 2 5 5  
Largest Element: 5  
No. of Occurances: 2
```

3. Given an unsorted dynamic array of size n, WAP to find and display the number of elements between two elements a and b (both inclusive).
E.g. Input : arr = [1, 2, 2, 7, 5, 4], a=2 and b=5, Output : 4 and the numbers are: 2, 2, 5, 4.

Source Code:

```
#include <stdio.h>  
#include <stdlib.h>  
  
void print_btwn_nums(int* ptr, int len);  
  
int main(){  
    int len;  
    printf("Enter array size : ");  
    scanf("%d", &len);  
  
    int *ptr = (int*) malloc(len * sizeof(int));  
  
    printf("Enter elements of array : ");  
    for (int i=0; i<len; i++){  
        scanf("%d", ptr+i);  
    }  
  
    print_btwn_nums(ptr, len);  
    free(ptr);  
    return 0;  
}  
  
void print_btwn_nums(int* ptr, int len){  
    int a, b, initial, final;
```

```

printf("a = ");
scanf(" %d", &a);
printf("b = ");
scanf(" %d", &b);

for (int i=0; i<len; i++){
    if (a == *(ptr+i)){
        initial = i;
    }
}

for (int i=initial; i<len; i++){
    if (b == *(ptr+i)){
        final = i;
    }
}
int count=0;
for (int i=initial; i<=final; i++){
    count ++;
    printf("%d, ", *(ptr+i));
}
printf("\nCount: %d\n", count);
}

```

Output:

```

Enter array size : 6
Enter elements of array : 1 2 2 7 5 4
a = 2
b = 5
2, 7, 5,
Count: 3

```

4. WAP to find out the second smallest and second largest element stored in a dynamic array.

Source Code:

```

#include <stdio.h>
#include <stdlib.h>

void sec_min_max(int* ptr, int len);

```

```
int main() {
    int len;
    printf("Enter array size: ");
    scanf("%d", &len);

    int *ptr = (int*) malloc(len * sizeof(int));

    printf("Enter elements of array: ");
    for (int i = 0; i < len; i++) {
        scanf("%d", ptr + i);
    }

    sec_min_max(ptr, len);
    free(ptr);
    return 0;
}
```

```
void sec_min_max(int* ptr, int len) {

    int first_min, sec_min, first_max, sec_max;
    int has_sec_min = 0, has_sec_max = 0;

    first_min = *ptr;
    first_max = *ptr;

    for (int i = 1; i < len; i++) {
        int val = *(ptr + i);

        if (val < first_min) {
            sec_min = first_min;
            has_sec_min = 1;
            first_min = val;
        } else if (val > first_min) {
            if (!has_sec_min || val < sec_min) {
                sec_min = val;
                has_sec_min = 1;
            }
        }

        if (val > first_max) {
            sec_max = first_max;
            has_sec_max = 1;
        }
    }
}
```

```

        first_max = val;
    } else if (val < first_max) {
        if (!has_sec_max || val > sec_max) {
            sec_max = val;
            has_sec_max = 1;
        }
    }
}

if (has_sec_min) {
    printf("Second smallest element: %d\n", sec_min);
} else {
    printf("No distinct second smallest element found.\n");
}

if (has_sec_max) {
    printf("Second largest element: %d\n", sec_max);
} else {
    printf("No distinct second largest element found.\n");
}
}

```

Output:

```

Enter array size: 5
Enter elements of array: 1 3 5 7 10
Second smallest element: 3
Second largest element: 7

```

5. WAP to store n student's information (i.e. student's roll no, name, gender, marks) of an educational institute and display all the data, for n students using array of structure.

Source Code:

```

#include <stdio.h>
#include <stdlib.h>

struct student {

```



```

    int roll;
    char name[50];
    char gender[10];
    float marks;
};

void input_students(struct student *s, int n);
void display_students(struct student *s, int n);

int main() {
    int n;
    printf("Enter number of students : ");
    scanf("%d", &n);

    struct student *s = (struct student*) malloc(n * sizeof(struct
student));

    input_students(s, n);
    display_students(s, n);

    free(s);
    return 0;
}

void input_students(struct student *s, int n) {
    for (int i = 0; i < n; i++) {
        printf("\nEnter details for student %d:\n", i + 1);
        printf("Enter Roll No: ");
        scanf("%d", &(s + i)->roll);
        printf("Enter Name: ");
        scanf("%s", (s + i)->name);
        printf("Enter Gender: ");
        scanf("%s", (s + i)->gender);
        printf("Enter Marks: ");
        scanf("%f", &(s + i)->marks);
    }
}

void display_students(struct student *s, int n) {
    printf("\nStudent Information\n");
    for (int i = 0; i < n; i++) {
        printf("\nStudent %d:\n", i + 1);
        printf("Roll No : %d\n", (s + i)->roll);
    }
}

```

```

        printf("Name      : %s\n", (s + i)->name);
        printf("Gender    : %s\n", (s + i)->gender);
        printf("Marks     : %.2f\n", (s + i)->marks);
    }
}

```

Output:

```

Enter number of students : 2

```

```

Enter details for student 1:

```

```

Enter Roll No: 101

```

```

Enter Name: Rahul

```

```

Enter Gender: Male

```

```

Enter Marks: 88.5

```

```

Enter details for student 2:

```

```

Enter Roll No: 102

```

```

Enter Name: Ananya

```

```

Enter Gender: Female

```

```

Enter Marks: 94.0

```

```

Student Information

```

```

Student 1:

```

```

Roll No : 101

```

```

Name    : Rahul

```

```

Gender  : Male

```

```

Marks   : 88.50

```

```

Student 2:

```

```

Roll No : 102

```

```

Name    : Ananya

```

```

Gender  : Female

```

```

Marks   : 94.00

```

6. WAP to declare one distance structure (with members kilometer and meter) and create the variables for addition of two distances using Pointers to structure.

10km500m--D1 - > ptr1

21km600m--D2 - > ptr2

Now use the third variable D3 and its pointer - >ptr3

Source Code:

```

#include <stdio.h>

struct distance {
    int kilometer;
    int meter;
};

void add_distance(struct distance *ptr1, struct distance *ptr2, struct
distance *ptr3);
void display_distance(struct distance *ptr);

int main() {
    struct distance D1, D2, D3;
    struct distance *ptr1 = &D1;
    struct distance *ptr2 = &D2;
    struct distance *ptr3 = &D3;

    printf("Enter first distance (kilometer and meter): ");
    scanf("%d %d", &ptr1->kilometer, &ptr1->meter);

    printf("Enter second distance (kilometer and meter): ");
    scanf("%d %d", &ptr2->kilometer, &ptr2->meter);

    add_distance(ptr1, ptr2, ptr3);

    printf("Sum of distances: ");
    display_distance(ptr3);

    return 0;
}

void add_distance(struct distance *ptr1, struct distance *ptr2, struct
distance *ptr3) {
    ptr3->kilometer = ptr1->kilometer + ptr2->kilometer;
    ptr3->meter = ptr1->meter + ptr2->meter;

    if (ptr3->meter >= 1000) {
        ptr3->kilometer += ptr3->meter / 1000;
        ptr3->meter = ptr3->meter % 1000;
    }
}

```

```
void display_distance(struct distance *ptr) {  
    printf("%d km %d m\n", ptr->kilometer, ptr->meter);  
}
```

Output:

```
Enter first distance (kilometer and meter): 10 500  
Enter second distance (kilometer and meter): 21 600  
Sum of distances: 32 km 100 m
```